

Phase 2: Syntax Analysis :-

Syntax Analysis is the second phase of a compiler.

It takes the **tokens** produced by **lexical Analysis** and checks whether they are arranged according to the grammar rules of the programming language.

The component that performs syntax analysis is called a **parser**.

In simple words;

Syntax Analysis checks whether the structure of a program is correct or not.

for eg, in English:

"Akash is going to school." ✓ correct

The words are arranged correctly.

But:

"Akash school to going is." ✗ wrong

The words exist, but their arrangement is wrong.

Similarly, a parser checks the correct arrangement of tokens in a programming language.

→ Position of syntax Analysis in Compiler :-

We already studied lexical Analysis in phase 1.

Source Program



{ Lexical Analysis
(Scanner) }



Tokens



{ Syntax Analysis
(Parser) }



Parse Tree



Semantic Analysis

Lexical Analyzer → Tokens → Parser → Parse Tree

→ Why do we need Syntax Analysis?

A programming language has certain grammar rules.

for eg;

an assignment statement can be written as:

identifier = expression;

$x = 10;$

is syntactically correct.

But:

$= x 10;$

does not follow the grammar.

The parser detects this problem.

Eg for Syntax Analysis:

Consider:

$a = b + c;$

Step 1:

lexical Analyzer produces token :-

a → Identifier
 = → Assignment operator
 b → Identifier
 + → operator
 c → Identifier
 ; → Separator

Step 2:

Parser checks their arrangement:

it check whether:

identifier = Expression;
 is a valid Structure.

~~Since it is invalid,~~

a = b + c;
 is accepted.

Eg; for syntax error :-

Consider :

① $a = + b ;$

② $a = b + ;$

here, tokens are:

$a \rightarrow$ Identifier

$= \rightarrow$ operator

$b \rightarrow$ Identifier

$+ \rightarrow$ operator

$; \rightarrow$ separator

The tokens are themselves are valid

but their arrangement is incorrect.

So parser report :

Syntax error.

Note:-

Lexical Analyzer - "Are these valid tokens"

Syntax Analyzer - "Are these tokens arranged correctly".

→ What is Grammar?

A grammar is a set of rules that tells us how valid statement or expression can be formed.

for eg;

Expression $\rightarrow$ identifier + identifier

According to this rule;

$a + b$
is valid

But:

$+ a b$

is not valid according to this particular grammar

We mainly study Context-free Grammar (CFG)
for Syntax Analysis.

→ Parser :-

A parser is a component of the compiler that takes tokens and checks whether they follow the grammar of the programming language.

It usually produces parse tree.

Tokens



Parser



Correct → Parse tree

└── Incorrect → Syntax Error

→ Main function of parser :-

The parser performs several important tasks.

1.) Check Syntax:

It checks whether the program follows grammar rules.

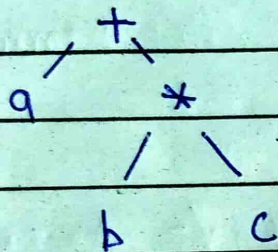
2.) Create parse tree:

The parser represents the structure of the program using a parse tree.

for eg;

$$a + b * c$$

Because multiplication has higher precedence than addition.



This shows that:

$$b * c$$

is performed before:

$$a + \dots$$

3.7 Reports Syntax errors :-

Eg;

if (x > 10 {

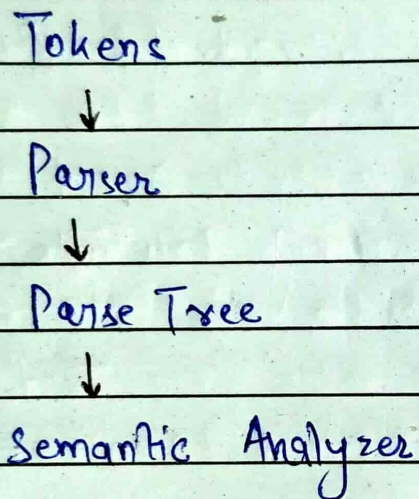
}

There is a missing).

The parser detects this and report a syntax error.

4. Passes Structure to the next phase :-

After successful parsing, the resulting Structure is passed to semantic Analysis



→ Parse Tree :-

A parse tree is a tree representation of the syntactic structure of a program according to a grammar.

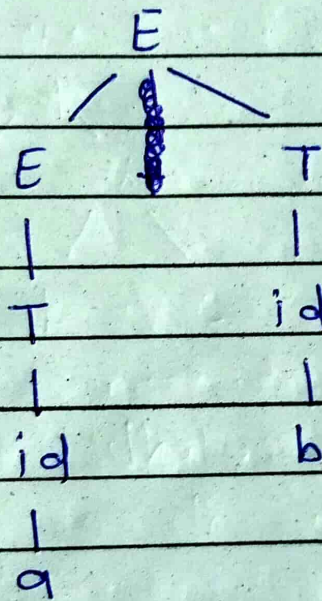
let's take a simple grammar:

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow id$$

for :

$a+b$

a simplified parse tree can be represented as:



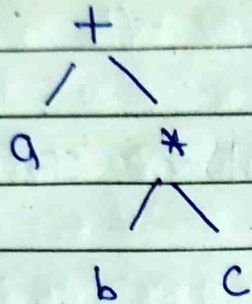
The tree shows how the expression is generated from the grammar.

→ Why is parse tree important?
The parse tree helps the compiler understand the structure of the program.

for eg;

$a+b+c$

the tree tells us that :



therefore;

$b * c$

calculated first-

then

$a + (b * c)$

→ Syntax Analysis and Errors :-

There are different types of error in a compiler.

Syntax Analysis mainly deals with **Syntax error**.

Eg 1: Missing Bracket:

```
if (a > b {
```

Correct:

```
if (a > b) {
```

Eg 2: Missing Semicolon:

```
int a = 10
```

Correct:

```
int a = 10;
```

Eg 3: Incorrect Expression:

```
a + * b
```

The arrangement of operation is incorrect under the usual expression grammar.

Complete Example :-

Suppose the source code is :

int x = 10 + 20 ;

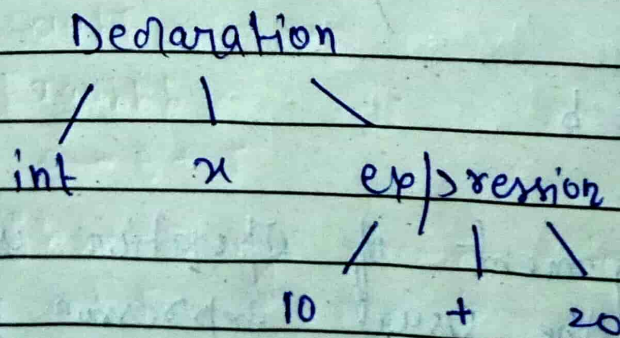
Lexical Analysis :

It produces :

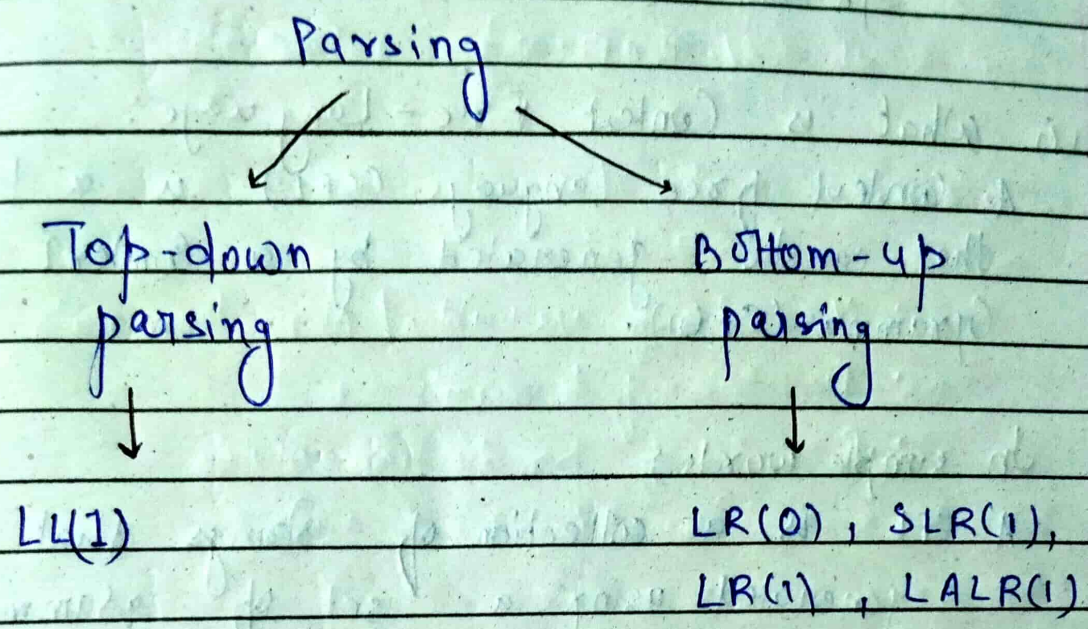
int	→	keyword
x	→	identifier
=	→	operator
10	→	constant
+	→	operator
20	→	constant
;	→	separator

Syntax Analysis :-

The parser checks whether the tokens form a valid declaration / assignment.



→ Types of Parsing :-



We will learn later in detail we study first CFG.

→ Context-Free Language and Context-free Grammar (CFG):

(i) What is Context-Free-Language:

A Context free language (CFL) is a language that can be generated by a Context-free-Grammar (CFG).

In simple words;

A CFL is a collection of strings that can be generated using a set of grammar rules where the left side of every rule contain at only one non-terminal.

Eg;

$$S \rightarrow aSb$$
$$S \rightarrow \epsilon$$

This grammar generates:

ab

aabb

aaabbb

...

∴ these strings belongs to CFL

Page No.
 Date
 why do we study CFL?
In compiler design, programming language have complex structure.

Eg;
 $a = b + c;$

The compiler needs to understand that:

a is a variable
 $=$ is an assignment operator
 $b+c$ is an expression.

Regular expression and finite automata are useful for **tokens**, but they are not powerful enough to describe the complete grammar structure of programming languages.

∴ We use **CFG**

→ What is Context-free Grammar (CFG)?
A context free Grammar (CFG) is a formal grammar used to describe the syntax or structure of languages, especially programming languages.

A CFG is represented by four components:

$$G = (V, T, P, S)$$

where,

- V = set of variables / Non-terminals
- T = set of terminals
- P = set of production rules
- S = Start symbol

(i) Variables / Non-terminals :-

A symbol that can be further expanded.

Usually, Capital's letter are used:

S

E

A

etc.

Eg;

$$E \rightarrow E + T$$

here

E and T are non-terminals

(ii) Terminals:

Terminals are actual symbol or character that appear in final string.

Eg;

a

b

(

)

+

*

id

etc

for eg;

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

here,

$id \rightarrow \text{Terminal}$

(iii) Production Rules:-

defines how non-terminals can be replaced.

Eg; $E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

iv, Start symbol:

The start symbol is the symbol from which we begin generating a string.

Eg;

$$S \rightarrow aSb$$

$$S \rightarrow \epsilon$$

here,

S is the start symbol

→ Why CFG is important in Compiler design:

CFG is mainly used to describe the syntax of programming language.

Eg;

$$a + b * c$$

Simplified grammar could be:

$$E \rightarrow E + T$$

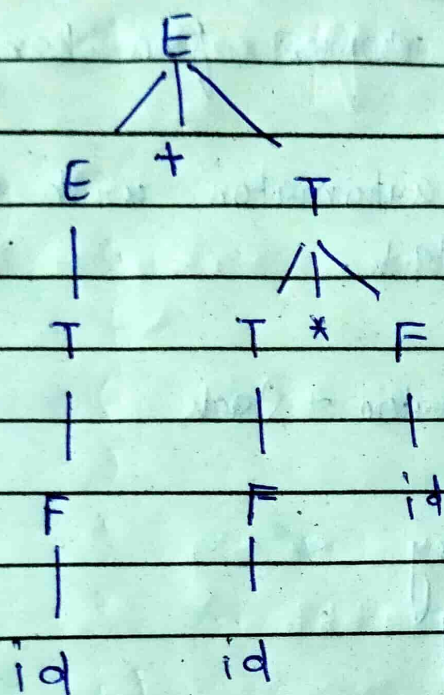
$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow id$$

The Structure :



a

b

c

Note: In FLAT/TOC we studied in detail about PDA.

→ Pushdown Automata (PDA):

A pushdown Automata (PDA) is a machine used to recognize context-free language (CFL).

It is like a finite automaton with an extra memory called a stack.

$$PDA = \text{finite automaton} + \text{stack}$$

- DFA/NFA → no memory
- PDA → has stack memory

→ Stack:

A stack works on:

LIFO

{ Last In First Out }

operation:

- push = Insert symbol
- pop = Remove top symbol

→ Why do we need PDA?

✓ We already studied Finite Automata (FA).

Finite automaton are good at recognizing regular language.

for eg: a^*

But finite automaton have a limitation:
 They have no unlimited memory to remember thing.

Consider this language:

$$L = \{ a^n b^n \mid n \geq 0 \}$$

It contains:

$\epsilon, ab, aabb, aaabbb, \dots$

here, number of a's must be equal to number of b's

so we need a stack.

→ Working of PDA:~

for eg: $a^n b^n$

Step 1:

• Read a's

• Push each a into stack

Step 2:

Read b's -

• pop one a for each b.

Step 3:

If stack become empty at end, it means.

accept

Eg:

for String : a a a b b b

Reading a:

push

a

a a

a a a

a
a
a

Reading b

pop

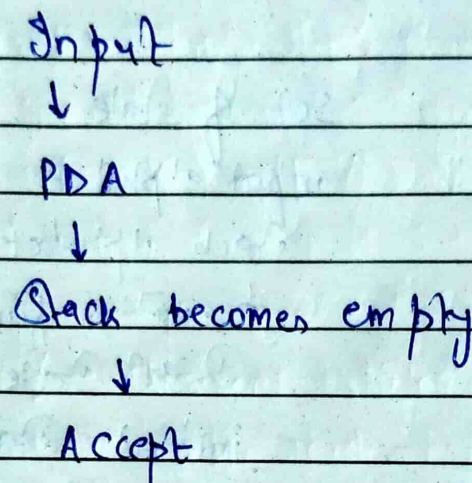
a a

a

ε

stack empty
accept

2. Acceptance by Empty Stack:
The PDA accepts when all of input has been processed and the stack becomes empty.



→ Biggest use of PDA is

Consider a C-program:

```
if (x) {  
    while (y) {  
        ==  
    }  
}
```

() { () { } }

There are nested parenthesis.

The PDA can use its stack.

Read : C

• push

Read :

push

Read :

push

Read :)

pop

Read :

push

Read :

pop

Read :

pop

At the end, the stack returns to its bottom conditions.

We will do there only one example, want more expt example please explore ~~Flat~~ Flat/Toc.

Note: ~

Transition

$$\delta(q, a, X) = (P, Y)$$

here,

$q \rightarrow$ current state

$a \rightarrow$ input read

$X \rightarrow$ top of the stack (Tos)

$P \rightarrow$ next state

$Y \rightarrow$ stack replacement.

State changes whenever:

Push $\leftrightarrow$ pop

push $\rightarrow$ skip

pop $\rightarrow$ skip

Eg:

Design a PDA to accept the following language by empty stack.

$$(0^n 1^n \mid n \geq 1)$$

⇒ let's generate string

0	0	0	1	1	1	ε
---	---	---	---	---	---	---

① $\delta(q_0, 0, z_0) = (q_0, 0z_0)$

② $\delta(q_0, 0, 0) = (q_0, 00)$

③ $\delta(q_0, 0, 0) = (q_0, 00)$

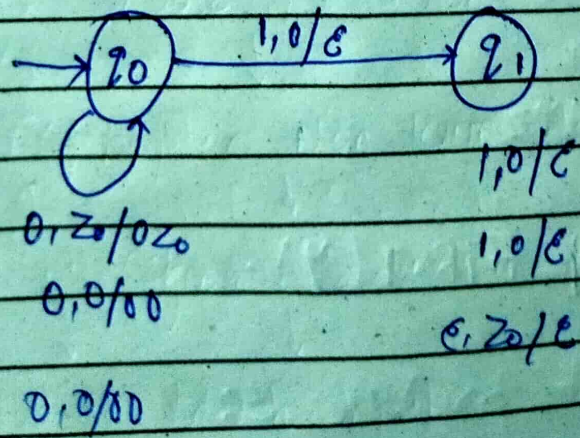
④ $\delta(q_0, 1, 0) = (q_1, 00)$

⑤ $\delta(q_1, 1, 0) = (q_1, 0z_0)$

⑥ $\delta(q_1, 1, 0) = (q_1, z_0)$

⑦ $\delta(q_1, \epsilon, z_0) = (q_1, z_0)$

PDA diagram:



Done